

THE UNIVERSITY OF
SYDNEY

ELEC5623 Assignment 1

A Closed-Loop Requirements-to-Test AI Agent for Autonomous Software
Testing

Wednesday 11 AM - 2 PM Group 4

Selected Track: Track A — AI Agent Product

Full Name | Student ID

Zheng Gong | 550444927

Jing Xiao | 540788617

Siyu Huang | 560528763

Xiangken Zhou | 550383659

Han Zhao | 560460784

Tutor Name: Mr Linghan Huang, Mr Haolin Jin

August 22, 2026

1. Executive Summary

Our project presents a Closed-Loop Requirements-to-Test AI Agent, an innovative product designed to revolutionise how software requirements are validated. The core challenge we address is the "hallucination" problem in AI-generated tests, where outputs appear plausible but are disconnected from the actual requirements or codebase.

Our agent solves this by implementing a traceable, feedback-driven workflow. It doesn't just generate tests; it analyses requirements, retrieves supporting evidence, executes the generated tests, and learns from the results to iteratively improve them. This ensures that each test is not only executable but also demonstrably linked to its source requirement and proven to run in a real environment.

The primary value to users—software developers and QA engineers—is a substantial reduction in manual testing effort combined with a marked increase in test reliability and coverage. By integrating requirement traceability, evidence grounding, and an automated refinement loop, our product delivers a final report that clearly maps tests back to requirements and highlights unresolved gaps, providing a comprehensive and trustworthy testing package. The system's performance will be quantified through a rigorous evaluation plan, focusing on its ability to reduce hallucination rates and achieve high mutation scores, proving its superiority over simpler, non-iterative approaches.

2. Introduction and Background

2.1 Background

Software testing is essential for quality assurance, yet translating natural-language requirements into executable tests remains a manual, time-consuming process. Developers and QA engineers must interpret specifications, design scenarios, implement tests, and validate outcomes—each step prone to inconsistency and oversight.

Large Language Models (LLMs) offer a promising avenue for automation by interpreting requirements and generating test code. Tools like GitHub Copilot and specialised AI testing platforms demonstrate this potential. However, syntactically plausible generated tests are not necessarily reliable; they may introduce unsupported behaviours, incorrect APIs, or assumptions misaligned with the original requirements.

As evidenced by prior work such as TestGen-LLM, only a subset of generated tests successfully build, pass reliably, and improve coverage. This highlights a critical gap: validation must be an integral part of any AI-driven test generation process, not an afterthought.

2.2 Existing Technical Approaches and Gap

Existing AI-assisted test generation falls into three categories: General-purpose coding assistants, such as GitHub Copilot, generate tests through developer interaction but rely heavily on users to validate and refine outputs. Specialised testing tools, such as Qodo, Diffblue Cover, Testim, offer more automated capabilities, yet often lack deep integration with requirements analysis and traceability. Research systems, such as ChatUniTest, TestGen-LLM, demonstrate the value of validating or repairing generated tests post-generation, but typically operate without a unified

workflow that connects requirement understanding, evidence grounding, and iterative execution feedback.

These developments show progress toward automation, but an opportunity remains: integrating requirement analysis, evidence grounding, test planning, executable generation, execution feedback, and iterative refinement within a single, traceable agentic workflow. This is the gap our proposed Requirements-to-Test AI Agent addresses.

3. Problem Statement and Motivation

3.1 Problem Statement

This project addresses the gap between natural-language software requirements and reliable, executable tests.

Existing LLM-based test generation can produce syntactically plausible tests, but these often contain unsupported assumptions, incorrect APIs, or expected behaviours not justified by the original requirements or project context. Ambiguous or incomplete requirements further compound this risk. Moreover, one-shot generation provides no mechanism to use real execution results for diagnosing and improving inadequate tests.

The central research question is: How can a GenAI agent transform software requirements into grounded and executable tests while using real execution feedback to identify and refine unreliable outputs?

The project focuses on three specific challenges: Grounding generated tests in available evidence (requirements, source code, testing knowledge); Maintaining traceability from requirements to tests; Using execution feedback to drive controlled refinement, rather than relying on repeated manual prompting.

3.2 Motivation for Generative AI and Agentic Design

Generative AI is suitable for this problem because software requirements are expressed in natural language and require semantic interpretation before translation into test scenarios and code. LLMs can support requirement analysis, test planning, and test generation while incorporating retrieved project context.

However, a standalone LLM is insufficient. An agentic architecture is essential to coordinate multiple stages and external resources: analysing requirements, retrieving evidence, inspecting project context, planning tests, invoking pytest, observing execution results, and deciding whether to refine or escalate to a human.

The proposed agent is therefore designed as a multi-step, tool-using closed-loop system rather than a generic chatbot. Its success will be measured through observable outcomes: test executability, requirement coverage, hallucination rate, and testing effectiveness.

4. Stakeholders and Requirements Engineering

4.1 Stakeholders and User Needs

The primary stakeholders are software developers and QA engineers, who require efficient generation of reliable and executable tests with clear traceability to software

requirements. Secondary stakeholders include product owners, who benefit from identifying ambiguous or insufficiently testable requirements, and team leads, who require consolidated evidence of requirement coverage and testing outcomes.

Stakeholder	Key Need
Software Developers	Reduce manual test design and generate executable tests aligned with project code
QA Engineers	Improve systematic test coverage, traceability and reliability
Product Owners	Identify ambiguous or insufficiently testable requirements
Team Leads	Obtain consolidated evidence of requirement coverage and testing outcomes

These needs are translated into the functional and non-functional requirements below.

4.2 Functional Requirements

ID	Functional Requirement	Priority
Proposer: Zheng Gong		
FR1	The system shall accept software requirements in supported document formats.	Must
FR2	The system shall extract and structure individual requirements for downstream analysis.	Must
FR3	The system shall identify ambiguous, incomplete or insufficiently testable requirements.	Must
Proposer:		
FR4	The system shall inspect relevant source-code context from a user-provided Python repository.	Must

FR5 The system shall retrieve relevant testing knowledge and Must project evidence using RAG.

FR6 The system shall generate a structured test plan for testable Must requirements.

Proposer:

FR7 The system shall generate pytest-compatible tests from the Must test plan and supporting evidence.

FR8 The system shall maintain traceability between requirements, Must supporting evidence and generated tests.

FR9 The system shall execute generated tests using pytest in a Must controlled environment.

Proposer:

FR10 The system shall capture and analyse test execution results Must and errors.

FR11 The system shall diagnose invalid or inadequate generated Must tests using requirement, project and execution evidence.

FR12 The system shall refine or extend generated tests based on Must execution feedback where appropriate.

Proposer:

FR13 The system shall re-execute refined tests within a bounded Must iteration process.

FR14 The system shall identify testable requirements without Should corresponding generated tests.

FR15 The system shall generate a final report summarising Must requirement-test mappings, execution results, coverage gaps and unresolved issues.

4.3 Non-Functional Requirements

ID	Category	Non-Functional Requirement
NFR1	Reliability	The system shall record all execution outcomes and shall not present unexecuted tests as validated.
NFR2	Traceability	Each generated test shall retain links to its source requirement and relevant supporting evidence.
NFR3	Performance	End-to-end workflow latency and test-execution time shall be measured for all benchmark runs and reported during evaluation.
NFR4	Usability	Users shall be able to inspect requirements, generated tests, execution results, coverage gaps and agent explanations through a consistent report or interface.
NFR5	Safety	Generated test code shall execute only within the designated sandbox or testing environment.
NFR6	Reproducibility	Model versions, prompts/configurations, retrieval settings and evaluation outputs shall be recorded sufficiently to reproduce experimental runs.

4.4 Assumptions and Constraints

Type	Description
Assumption 1	Input projects contain sufficient source code and dependency information for tests to be executed.
Assumption 2	Requirements are provided in supported formats and contain sufficient information for at least some behaviours to be tested.

Assumption 3 Evaluation repositories can be safely executed in the designated sandbox environment.

Assumption 4 The selected LLM API and required testing tools remain available during development and evaluation.

Constraint 1 The project must be designed, implemented and evaluated within one semester.

Constraint 2 The initial prototype is limited to Python projects and pytest.

Constraint 3 The prototype primarily targets functional and unit testing; performance, penetration and large-scale end-to-end testing are outside the core scope.

Constraint 4 System performance and operational cost depend partly on the selected LLM and API availability.

4.5 Key Acceptance Criteria

Requirement	Acceptance Criterion
FR2 – Requirement Extraction	For benchmark specifications with manually established reference requirements, the system shall extract structured requirements that can be evaluated using precision, recall and F1.
FR3 – Ambiguity Detection	For benchmark requirements labelled as ambiguous or insufficiently specified, the system shall identify and flag the uncertainty rather than generate unsupported expected behaviour.
FR7 – Test Generation	For supported benchmark projects, the system shall generate pytest-compatible candidate tests without manual rewriting before execution.
FR8 – Traceability	Each evaluated generated test shall retain a traceable link to its source requirement and relevant supporting evidence.

FR9 – Test Execution	The system shall submit generated tests to pytest and record the pass, fail or error status for each executed test.
FR12/FR13 – Refinement and Re-execution	When an initial test is diagnosed as invalid or inadequate, the system shall perform at least one evidence-supported refinement and re-execution attempt, unless the configured stopping condition is reached.
FR14 – Requirement Coverage	The system shall identify testable requirements for which no corresponding generated test exists.
FR15 – Final Report	The system shall produce a final report containing requirement-test mappings, execution outcomes, identified coverage gaps and unresolved issues.

5. Proposed Product and Innovation

5.1 Product Concept

The proposed product is a closed-loop Requirements-to-Test AI Agent that transforms software requirements and project context into executable tests and iteratively improves them using real execution feedback.

The agent follows an end-to-end workflow: Parse Requirements → Analyse Testability → Retrieve Evidence → Plan Tests → Generate Tests → Execute Tests → Analyse Results → Refine → Re-execute → Report

Unlike one-prompt LLM test generation, the proposed system integrates requirement understanding, project context, testing knowledge and real test execution within a single agentic workflow. Each generated test remains traceable to its source requirement and supporting evidence, enabling the system to identify uncovered requirements and produce a consolidated testing report.

The initial prototype will focus on Python projects using pytest, with functional and unit testing as the core testing scope.

5.2 Product Innovation and Differentiation

The proposed product differs from a generic chatbot or single-prompt test generator through three key capabilities.

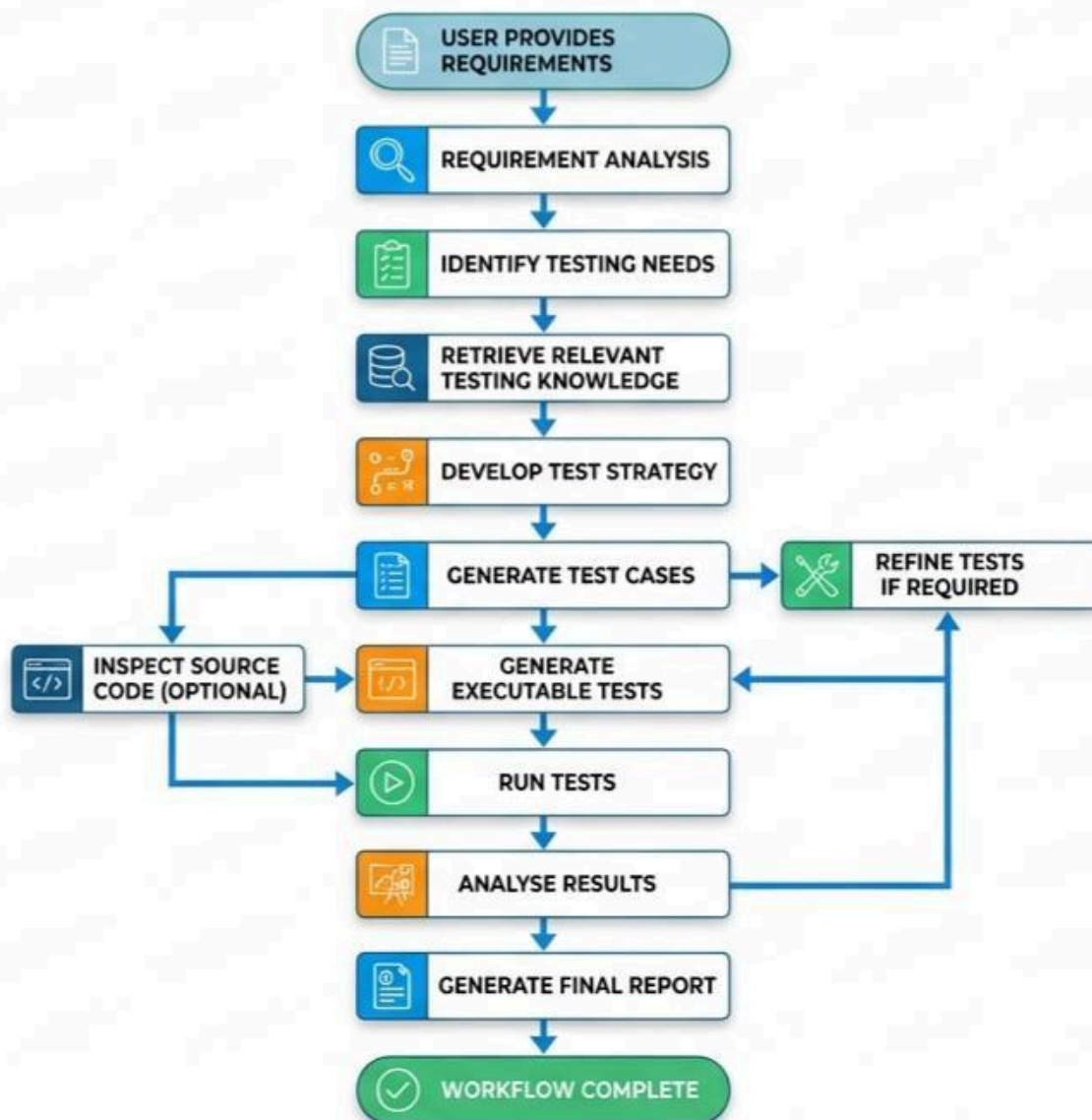
1. **Closed-loop validation.** Generated tests are executed in a real testing environment rather than accepted directly as LLM outputs. Execution results are returned to the agent, allowing it to diagnose failures, refine inadequate tests and re-execute them when appropriate.
2. **Evidence-grounded generation and traceability.** The agent uses requirements, relevant source-code context and retrieved testing knowledge to support test planning and generation. Generated tests retain links to their

source requirements and supporting evidence, reducing unsupported assumptions and enabling requirement-level coverage analysis.

3. **Controlled autonomy.** The agent operates autonomously when sufficient evidence is available but does not invent expected behaviour when requirements are ambiguous or evidence is insufficient. Such cases are flagged for human review, while iteration limits prevent uncontrolled refinement.

Together, these capabilities shift the product from single-shot AI test generation toward an evidence-grounded, traceable and closed-loop software testing agent.

5.3 User Workflow



5.4 AI Components

LLM reasoning and planning: The LLM interprets requirements, selects appropriate actions, generates tests, and analyses intermediate results.

Agent Runtime / Orchestration: The runtime maintains the workflow and state, manages tool calls, and passes tool results back to the LLM.

RAG: A retrieval system provides relevant testing knowledge to the LLM rather than requiring the model to rely solely on its pretrained knowledge.

Memory / State: The agent maintains intermediate information such as analysed requirements, retrieved documents, generated tests, tool outputs, and execution results throughout the workflow.

Tool Calling: The agent can invoke external capabilities such as file parsing, source-code inspection, Git repositories, pytest/JUnit, and report generation.

6. Proposed Methodology and System Design

6.1 Planned development approach

The project will adopt an iterative and incremental development approach. Rather than attempting to implement the complete autonomous agent at once, the system will be developed in several stages, with each stage providing a testable increment.

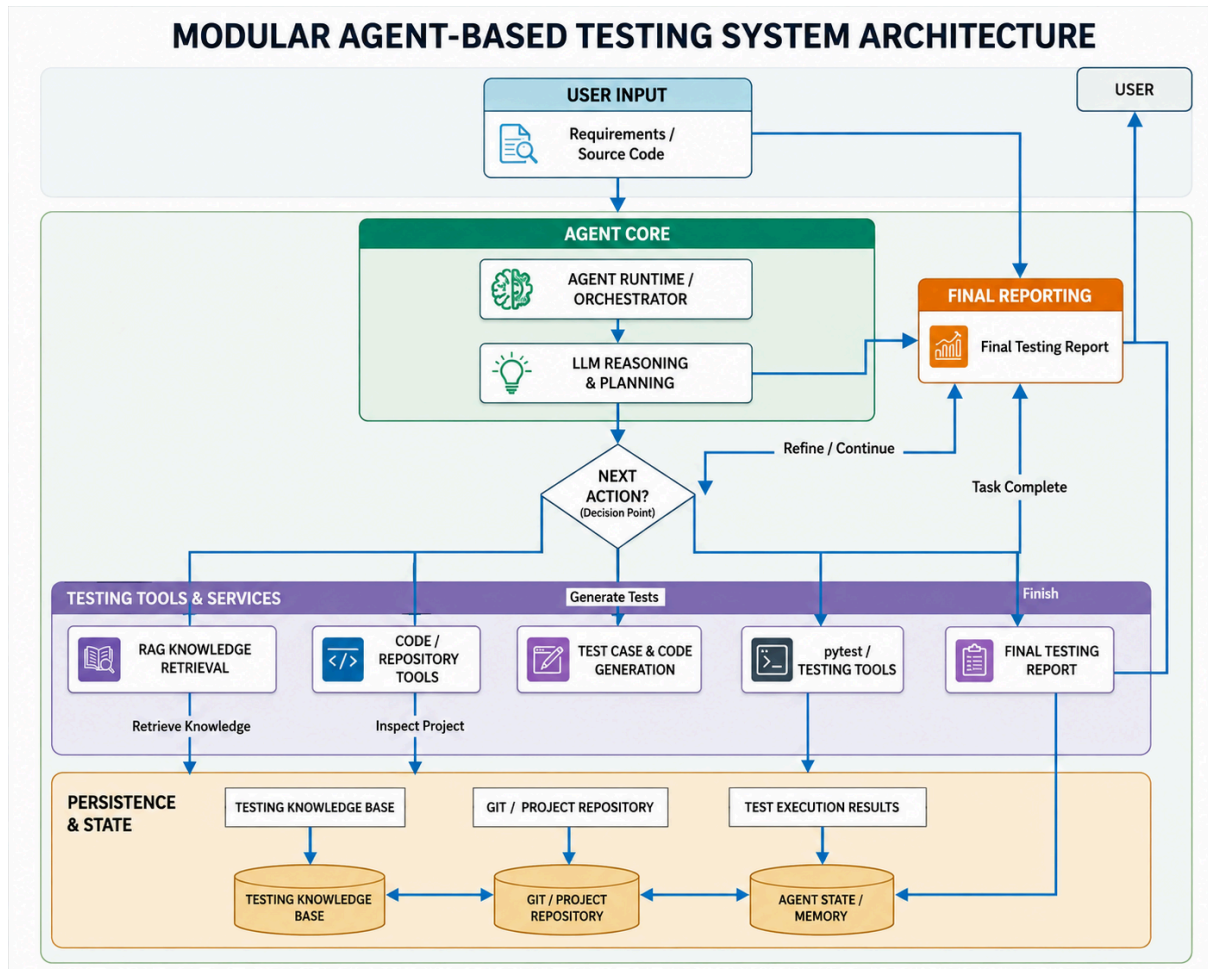
The planned development stages are:

1. **Requirements and test-case generation:** Develop the basic LLM pipeline for analysing requirements and generating structured test cases.
2. **Knowledge retrieval:** Add a software-testing knowledge base using RAG to provide testing techniques and domain knowledge.
3. **Agent workflow:** Introduce an Agent Runtime that maintains state and allows the LLM to determine the next action.
4. **Tool integration:** Add tools for file access, source-code inspection, and automated test execution.
5. **Feedback and refinement:** Enable the agent to analyse test results and iteratively improve generated tests.
6. **Evaluation:** Compare the complete agent against a conventional single-prompt LLM baseline.

This approach allows individual components to be tested independently before they are integrated into the complete agent.

6.1 System Architecture and Workflow

The proposed system follows a closed-loop agentic workflow that connects requirement understanding, evidence grounding, test planning, executable test generation, real test execution, diagnosis and refinement. The architecture integrates an LLM-based reasoning component with source-code inspection, retrieval-augmented generation (RAG), external testing tools and persistent workflow state. Figure 1 presents the overall architecture and information flow of the proposed system.



6.2 Requirement Understanding and Grounding

The agent first converts the input specification into structured, testable requirements and identifies ambiguous or insufficiently specified behaviours. It then inspects relevant source-code context and uses RAG to retrieve supporting information from project files, testing knowledge and framework documentation.

Retrieved evidence is linked to subsequent testing decisions to support requirement-to-test traceability. Where expected behaviour cannot be justified by the available evidence, the agent records the uncertainty rather than generating an unsupported test oracle.

6.3 Test Planning and Generation

Using the structured requirements and retrieved evidence, the agent creates a test plan covering relevant scenarios such as normal behaviour, invalid inputs, boundary conditions and negative cases. Appropriate testing techniques, including equivalence partitioning and boundary value analysis, may be selected where applicable.

The LLM then converts the plan into pytest-compatible tests using the inspected project interfaces and dependencies. Each generated test remains linked to its source requirement and supporting evidence.

6.4 Execution, Diagnosis and Closed-Loop Refinement

Generated tests are executed using pytest in a controlled environment. The system records execution results, errors and available coverage information and returns this feedback to the agent for diagnosis.

When a generated test is invalid or inadequate, the agent determines whether refinement is required based on requirement, source-code and execution evidence. It may revise or extend the test and execute it again, forming the closed loop: Generate → Execute → Diagnose → Refine → Re-execute

If the evidence instead suggests a potential implementation defect or unresolved requirement ambiguity, the issue is reported rather than automatically modifying the test. A configurable iteration limit prevents uncontrolled refinement.

6.5 Tools, Data Sources and Integrations

The prototype will integrate a limited set of tools and data sources to keep the project feasible within the semester scope.

Component	Proposed Resource / Purpose
Requirements	<i>PDF/Markdown specifications</i>
Project context	<i>User-provided Python repository</i>
Testing knowledge	<i>Curated knowledge base accessed through RAG</i>
AI component	<i>LLM API for analysis, planning, generation and diagnosis, Agent Runtime / Orchestration, RAG</i>
Test execution	<i>Pytest, (JUnit)</i>
Coverage	<i>pytest-compatible coverage tooling</i>
Environment	<i>Controlled sandbox/container</i>
Workflow state	<i>Requirements, evidence, tests and execution/refinement history</i>

The initial prototype will focus on Python projects, pytest and functional/unit testing. Additional languages, testing frameworks and large-scale testing environments are outside the core semester scope.

7. Business and Product Analysis

7.1 Market Need and Value Proposition

The initial target users are developers and QA engineers working on Python projects with documented functional requirements. Their current workflow involves interpreting requirements, designing and implementing tests, checking execution results and maintaining traceability. AI tools can assist with test generation, but users must still determine whether generated tests are executable and supported by the requirements.

The proposed agent addresses this problem through a single requirements-to-test workflow. Its value proposition is to generate traceable and executable pytest tests while using project evidence and execution feedback to reduce unsupported behaviour and manual refinement. Expected benefits include reduced test-design effort, earlier identification of ambiguous or uncovered requirements, and improved

reliability of generated tests. These benefits will be validated using the evaluation metrics in Section 8 rather than assumed in advance.

7.2 Competitor and Alternative Analysis

Existing alternatives address parts of the testing workflow but differ in their primary focus. GitHub Copilot supports developer-prompted unit-test generation from selected code. Qodo provides code-aware test generation, including Python and pytest support, but its documented workflow primarily centres on code components and pull-request changes. Diffblue Cover automates the generation and validation of Java unit tests, while Testim focuses mainly on recorded web and mobile UI testing.

Alternative	Main Focus	Limitation Relative to the Proposed Agent
GitHub Copilot	<i>General code and test assistance</i>	<i>Requirement traceability is not its central workflow</i>
Qodo	<i>Code- and change-based test generation</i>	<i>Primarily code-first rather than requirement-first</i>
Diffblue Cover	<i>Automated Java unit-test generation</i>	<i>Java-specific and code-centred</i>
Testim	<i>Web and mobile UI automation</i>	<i>Different testing level and output format</i>
Proposed Agent	<i>Requirements, evidence and execution feedback</i>	<i>Narrow initial scope and effectiveness not yet validated</i>
itHub Copilot	<i>General code and test assistance</i>	<i>Requirement traceability is not its central workflow</i>

The proposed agent is differentiated by integrating requirement analysis, evidence grounding, requirement-to-test traceability, pytest execution, explicit uncertainty handling and bounded refinement. Its intended advantage is therefore the complete and inspectable workflow, rather than any single test-generation capability.

7.3 Technical and Operational Feasibility

The prototype is feasible because it combines established technologies: an LLM API for requirement analysis and test generation, RAG for evidence retrieval, Python source-code inspection and pytest for test execution. Development risk is limited by restricting the MVP to Python repositories, PDF or Markdown requirements, functional and unit testing, and a controlled sandbox.

The main dependencies are LLM availability, retrieval quality and reproducible project environments. Missing dependencies, ambiguous requirements and environment-specific failures will be reported as unresolved issues instead of being treated as successful test generation. The modular architecture also allows

requirement analysis, retrieval, generation, execution and refinement to be developed and evaluated independently.

7.4 Costs and Operational Considerations

The main operational costs are LLM tokens, retrieval or embedding operations and test-execution resources. Closed-loop refinement increases these costs, so the prototype will use compact retrieval context, caching and a configurable iteration limit. Token usage, latency, execution time and refinement count will be recorded during evaluation.

Generated tests will execute only in a restricted sandbox. Public, synthetic or approved repositories will be used, and only relevant source-code context should be sent to external models. The initial product should operate as a developer-facing support tool with human review, rather than as an autonomous release gate.

7.5 Feasibility Analysis

This section assesses whether the closed-loop requirements-to-test AI agent can be completed within one semester under project assumptions and constraints.

7.5.1 Technical Feasibility

The prototype builds on existing mature open-source components without building foundation models from scratch. LangGraph handles agent orchestration, Chroma supports RAG knowledge retrieval, pytest enables automated test execution, and Docker provides an isolated sandbox satisfying safety requirement NFR5. Lightweight local storage persists agent state and test metadata.

Key risks are mitigated through engineering measures: LLM hallucinations are reduced by evidence-based prompts and maximum refinement iteration limits; RAG caching lowers token overhead; ambiguity detection prevents the agent from inventing test oracles for unclear requirements. The work depends on continued availability of external LLM APIs (Assumption 4). Overall, the project is technically achievable.

7.5.2 Operational Feasibility

Inputs are requirement documents and Python repositories; outputs are executable pytest tests and traceable HTML reports compatible with existing developer and QA workflows. A full graphical UI is out-of-scope; the prototype uses command-line operation to reduce development load. The system serves only as an assistive tool and does not replace human review of test outputs.

7.5.3 Economic Feasibility

Standard workstations are sufficient for development. Main variable costs stem from LLM API tokens and embedding calls. Cost-control tactics include bounded iterations, result caching and limited context transmission. Student API quotas meet prototype experimental needs. Commercial pricing models are reserved for potential future releases, as this project focuses solely on technical validation.

7.5.4 Constraint Feasibility

Hard constraints restrict scope to Python-pytest unit and functional testing. Other testing types and multi-language support are excluded. To ease schedule pressure, an MVP-first incremental strategy is followed: the core closed-loop workflow and quantitative evaluation are mandatory, while cosmetic interface features are optional and can be omitted if time is limited.

Conclusion: Given the defined scope, assumptions and risk-mitigation approaches, the prototype can be delivered within one semester.

8. Evaluation Plan

8.1 Evaluation Design and Test Scenarios

The proposed agent will be evaluated on representative Python projects containing software requirements, source code and known testable behaviours. The evaluation will assess whether the system can generate executable and requirement-aligned tests, improve testing effectiveness through closed-loop refinement, and avoid unsupported assumptions.

Test scenarios will include complete requirements, ambiguous or incomplete requirements, boundary and exception cases, and API/code traps where plausible but unsupported behaviours could lead to hallucinated tests. A hallucination is defined as a test decision or expected behaviour that cannot be supported by the provided requirements, project context or retrieved evidence.

Where feasible, benchmark projects will include known or seeded defects to evaluate whether generated tests can detect actual implementation faults.

8.2 Baselines

The full closed-loop agent will be compared against simpler configurations using the same project inputs:

Baseline	Configuration
B0: Direct LLM	<i>Generates tests directly from requirements without retrieval, execution feedback or refinement</i>
B1: Grounded LLM	<i>Generates tests using retrieved requirement and project evidence, but without closed-loop refinement</i>
B2: Full Agent	<i>Uses grounding, planning, test execution, diagnosis and iterative refinement</i>

This comparison will assess whether evidence grounding and closed-loop execution provide measurable improvements over conventional single-prompt test generation.

8.3 Evaluation Metrics

The evaluation will use quantitative metrics aligned with the project's core functional and non-functional requirements.

Metric	Purpose
Requirement Coverage	<i>Percentage of testable requirements linked to at least one generated test</i>
Test Execution Success Rate	<i>Percentage of generated tests that execute successfully in pytest</i>
Mutation Score	<i>Percentage of seeded or generated mutants detected by the test suite</i>
Hallucination Rate	<i>Percentage of generated tests containing unsupported requirements, APIs or expected behaviour</i>
End-to-End Completion Rate	<i>Percentage of evaluation tasks completed from requirement analysis through final reporting</i>

Where relevant, secondary measures such as code coverage, refinement success, latency and token/API cost will also be recorded. Results before and after refinement will be compared to determine whether the closed-loop process improves test quality rather than merely increasing the number of generated tests.

8.4 Qualitative Evaluation

A small qualitative evaluation will complement the quantitative results. Developers, QA practitioners or suitably experienced software-engineering participants may review selected system outputs using a structured rubric covering requirement relevance, correctness, traceability, usefulness of explanations and usefulness of ambiguity warnings.

The qualitative evaluation will help identify limitations that automated metrics may not capture, particularly whether generated tests and reports are understandable and practically useful to intended users.

9. Risks, Ethics and Responsible AI

Risk	Impact	Mitigation
Unsupported or hallucinated behaviour	<i>Incorrect tests and false confidence</i>	<i>Requirement grounding, evidence traceability and hallucination evaluation</i>
Generated test does not execute	<i>Reduced automation reliability</i>	<i>Automated pytest execution and structured refinement</i>
Agent misclassifies a software defect as a test-generation error	<i>Incorrect modification of a valid test</i>	<i>Preserve execution evidence and distinguish test errors from suspected SUT defects</i>
Ambiguous requirements	<i>Invented expected behaviour</i>	<i>Ambiguity detection and explicit uncertainty/clarification states</i>
Infinite refinement	<i>Excessive latency and cost</i>	<i>Configurable maximum refinement iterations</i>
Unsafe generated code	<i>Security risk</i>	<i>Execute generated code only in an isolated sandbox/container</i>
Sensitive source code sent to external models	<i>Privacy/IP risk</i>	<i>Use public or approved evaluation repositories and minimise transmitted context</i>

LLM/API variability	<i>Reduced reproducibility</i>	<i>Record model version and configuration and repeat selected experiments</i>
Evaluation benchmark bias	<i>Overestimated performance</i>	<i>Use multiple projects and requirement types and report failure cases</i>

Human oversight remains important. The system is intended as a software-testing support and automation tool rather than an authority that independently certifies software correctness or safety.

10. Semester Plan and Team Responsibilities

The development will follow an incremental sequence so that a working baseline exists before more advanced agent capabilities are added.

Period	Main Activities	Deliverable	Responsible Member(s)
Week 3	<i>Set up dev environment; finalise Architecture docs, dev tech stack; literature & competitor environment, review; define architecture & agent data-model spec, B0 state; build B0 direct-LLM baseline</i>	<i>baseline pipeline</i>	<i>All members</i>
Week 4	<i>Develop input-parsing module; Input-processing parse requirements, detect component for ambiguity; integrate Git for requirement & source-code loading source-code ingestion</i>		<i>All members</i>
Week 5	<i>Implement RAG retrieval pipeline; Functional RAG pipeline build testing knowledge-base; add and populated testing provenance traceability for retrieved knowledge base evidence</i>		<i>All members</i>
Week 6	<i>Develop core agent runtime: Agent scheduling loop implement agent orchestration, with task-planning LLM-based reasoning and planning capability logic; build tool-calling interfaces for invoking RAG and Git components</i>		<i>All members</i>
Week 7	<i>Develop test-case generation Module generating module; generate pytest code from traceability-annotated evidence; create requirement-test pytest test code traceability links</i>		<i>All members</i>

- Week 8** *Integrate pytest with Docker Isolated test-execution sandbox; capture test-execution sandbox; complete outputs; implement closed-loop end-to-end closed-loop diagnosis & refinement logic with workflow: Generate -> iteration-limit guardrails Execute -> Diagnose -> Note: Full system stability relies on Refine Week 9 buffer integration testing* *All members*
- Week 9** *Resolve system bugs; mitigate Stable agent looping behaviours, handle minimum-viable-product LLM output anomalies and sandbox prototype failures; perform full end-to-end integration testing using simple demonstration projects* *All members*
- Week 10** *Prepare evaluation assets; Benchmark dataset and construct benchmark datasets; evaluation automation implement automation scripts for scripts B0, B1, B2 baselines* *All members*
- Week 11** *Run comparative experiments; Raw experimental collect quantitative metrics including datasets, preliminary requirement coverage, hallucination analysed evaluation rate and test-execution success result rate; record supplementary measurements such as token consumption and wall-clock runtime; process and interpret experimental results* *All members*
- Week 12** *implement final HTML test-report generation; optimise report functional reporting visualisation.[Optional Extension] module, full project Lightweight command-line documentation and interactive interface; final-phase demonstration assets documentation, minor bug fixes, demo preparation and final report writing*

References

- [1]. Alshahwan, N., Chheda, J., Finogenova, A., Gokkaya, B., Harman, M., Harper, I., Marginean, A., Sengupta, S., & Wang, E. (2024). Automated Unit Test Improvement Using Large Language Models at Meta. FSE 2024.
- [2]. Chen, Y., Hu, Z., Zhi, C., Han, J., Deng, S., & Yin, J. (2024). ChatUniTest: A Framework for LLM-Based Test Generation. FSE 2024 Companion.
- [3]. Niu, C., Wu, Y., Zhu, J., Xu, S., Shum, K., Zhong, R., Song, J., & Zhang, T. (2024). RAGTruth: A Hallucination Corpus for Developing Trustworthy Retrieval-Augmented Language Models. ACL 2024.
- [4]. GitHub. (2026). GitHub Copilot Documentation: Generating Unit Tests.
- [5]. Qodo. (2026). AI-Powered Software Testing and Test Generation.
- [6]. Diffblue. (2026). Diffblue Cover — Autonomous Unit Test Generation.
- [7]. Tricentis. (2026). Testim — AI-Powered and Agentic Test Automation.